مقدمة عن النضرية

📌 القسم الأول: لماذا نحتاج نظرية البحث؟

🔷 الانتقال من الوصف إلى الحل

المرحلة	الوصف	الأدوات
Chapter 2	وصف المشكلة	Predicate Calculus, Logic
Chapter 3	**إيجاد الحل**	**State Space Search**

الفرق الجوهري:

- **قبل:** نعرف كيف نكتب "كل إنسان فانٍ"
- **الآن:** نعرف كيف نجد الطريق من بغداد إلى البصرة!

🔷 الأسئلة الأربعة الحاسمة 🔥

قبل كتابة أي خوارزمية بحث، يجب أن نستطيع الإجابة على:

1. الاكتمال (Completeness)

```text
السؤال: هل الخوارزمية مضمونة لإيجاد الحل إذا كان موجوداً؟
```

2. الإنهاء (Termination)

```text
السؤال: هل ستنتهي الخوارزمية أم ستدخل في حلقة مفرغة؟
```

المشكلة:

```text
A → B → C
↓       ↑
D ← ← ← ←
```

إذا لم نتتبع الحالات المزارة: A → B → C → D → A → ... (لا نهاية!)

3. الأمثلية (Optimality)

```text
السؤال: إذا وجدنا حلاً، هل هو الحل الأفضل؟
```

مثال:

```text
من Baghdad إلى Basra:
المسار 1: 450 km ← وجدته أولاً
المسار 2: 380 km ← الأمثل!
```

السؤال: هل الخوارزمية تضمن إيجاد المسار 2؟

---

4. التعقيد (Complexity)

```text
السؤال: كم يستغرق من وقت وذاكرة؟
```

المقياس	الوصف
Time Complexity	عدد العمليات المطلوبة
Space Complexity	الذاكرة المستخدمة

أمثلة:

- $O(n)$ ← ممتاز
- $O(n^2)$ ← مقبول
- $O(2^n)$ ← كارثة! (Exponential)
- $O(n!)$ ← مستحيل عملياً (Factorial)

🔷 لماذا هذه الأسئلة مهمة؟

بدون نظرية	مع نظرية
"جرّب وشوف!"	تحليل مسبق دقيق
قد ينجح، قد يفشل	نتائج مضمونة
لا نعرف متى ينتهي	نعرف الحدود الزمنية
حل عشوائي	حل أمثل

📌 القسم الثاني: الأساس الرياضي - نظرية البيان

🔷 قصة الاكتشاف: جسور كونيغسبرغ

المشكلة التاريخية (1736):

```text
مدينة Königsberg (الآن Kaliningrad)
- 4 مناطق
- 7 جسور تربطهم
السؤال: هل يمكن المشي عبر كل جسر مرة واحدة والعودة؟
```

محاولات الناس:

- جربوا آلاف المرات
- لم يستطيعوا
- **لكن:** لم يثبتوا الاستحالة!

حل أويلر (Euler):

الخطوة 1: التمثيل البياني

```text
المناطق → Nodes (عقد)
الجسور → Edges (أقواس)
```

الخطوة 2: التحليل الرياضي

```text
شرط الحل: كل عقدة يجب أن يكون عدد الأقواس المتصلة بها زوجياً
(أو عقدتان فقط فرديتان)
```

الخطوة 3: النتيجة

```text
في Königsberg:
- 3 عقد لها 3 أقواس (فردي)
- 1 عقدة لها 5 أقواس (فردي)
← 4 عقد فردية → مستحيل! ❌
```

الدرس المستفاد:

```text
التمثيل البياني > المحاولة والخطأ
النظرية الرياضية تكشف البنية الخفية للمشكلة
```

◆ العناصر الأساسية للرسم البياني

المكونان الأساسيان:

```text
Graph G = (V, E)
```

الرمز	الاسم	الوصف	التمثيل
V	Vertices (عقد)	النقاط/الحالات	● ○ ◆
E	Edges (أقواس)	الروابط/الانتقالات	→ —

أنواع الأقواس:

النوع	الوصف	الرمز	مثال
Directed	لها اتجاه	A → B	طريق أحادي الاتجاه
Undirected	بدون اتجاه	A — B	طريق ثنائي الاتجاه
Weighted	لها وزن/تكلفة	A → `abebooks` B	طريق بمسافة 5 km

◆ المصطلحات الهيكلية المتقدمة

1. المسار (Path)

تعريف: تسلسل من العقد حيث كل عقدة متصلة بالتالية

```text
مثال: A → B → C → D
الطول: 3 (عدد الأقواس)
```

أنواع المسارات:

النوع	الوصف
Simple Path	لا تتكرر فيه العقد
Cycle	يبدأ وينتهي بنفس العقدة
Acyclic	لا يحتوي على دورات

---

2. الاتصالية (Connectivity)

```text
Connected Graph: يوجد مسار بين أي عقدتين
Disconnected Graph: يوجد عقد منعزلة
```

مثال على Disconnected:

```text
A → B → C          X → Y
  (مجموعة 1)      (مجموعة 2)
```

أهمية: في Puzzle-8، فضاء الحالة **منفصل** (سنشرح لاحقاً)

---

3. الرسم الجذري (Rooted Graph)

تعريف: رسم بياني له **عقدة بداية واحدة** (Root)

```text
        ROOT
       /  |  \
      A   B   C
     /     \
    D       E
```

خصائص:

- كل مسار يبدأ من Root
- يمثل **بداية المشكلة**
- أساسي في خوارزميات البحث

4. الشجرة (Tree)

تعريف: رسم بياني حيث يوجد **مسار واحد فريد** بين أي عقدتين

شروط الشجرة:

1. ✅ لا توجد دورات (Acyclic)
2. ✅ كل عقدة (عدا الجذر) لها أب واحد
3. ✅ متصلة (Connected)

مثال على شجرة: ✅

```text
      A
     / \
    B   C
   /     \
  D       E
```

مثال على غير شجرة: ❌

```text
      A
     / \
    B - C  ← وصلة إضافية تخلق دورة
```

5. DAG (Directed Acyclic Graph)

تعريف: رسم بياني موجه **بدون دورات**

```text
   A
  / \
 B   C
  \ /
   D
```

خصائص:

- كل الأقواس لها اتجاه
- لا توجد دورات
- **مثال:** Tic-Tac-Toe (لا يمكن التراجع)

◆ العلاقات العائلية (Family Relations)

```text
        A (Grandparent)
       / \
      B   C (Parent)
     / \   \
    D   E   F (Children)
     \ /
      G (Grandchild)
```

المصطلح	الوصف	مثال
Parent	العقدة السابقة مباشرة	B هو parent لـ D
Child	العقدة اللاحقة مباشرة	D هو child لـ B
Siblings	عقد لها نفس Parent	D و E أشقاء
Ancestor	أي عقدة في المسار للأعلى	A جد لـ G
Descendant	أي عقدة في المسار للأسفل	G حفيد لـ A
Leaf	عقدة بدون أبناء	G ورقة (Leaf)

◆ درجة العقدة (Node Degree)

في Undirected Graph:

```text
Degree = عدد الأقواس المتصلة بالعقدة
```

في Directed Graph:

النوع	الوصف
In-degree	عدد الأقواس الداخلة
Out-degree	عدد الأقواس الخارجة

مثال:

```text
A → B → C
    ↓
    D
```

العقدة B:

- In-degree = 1 (من A)
- Out-degree = 2 (إلى C و D)
- Total degree = 3

📌 القسم الثالث: التعريف الرسمي لفضاء الحالة

🔷 الصيغة الرياضية

```text
State Space = [N, A, S, GD]
```

tuple-4 (رباعية مرتبة):

الرمز	الاسم	النوع	الوصف
N	Nodes	Set	مجموعة جميع الحالات الممكنة
A	Arcs	Set	مجموعة العمليات/الانتقالات
S	Start	Subset of N	حالة/حالات البداية
GD	Goal Description	Predicate/Function	وصف الهدف المطلوب

🔷 شرح تفصيلي لكل عنصر

N: مجموعة الحالات (Nodes)

تعريف: مجموعة **جميع** الحالات التي يمكن أن تحدث في المشكلة

أمثلة:

المشكلة	N (الحالات)	الحجم
Tic-Tac-Toe	كل ترتيبات X و O	~5,478 حالة قانونية
Puzzle-8	كل ترتيبات 1-8 + فراغ	9!/2 = 181,440
Checkers	كل أوضاع القطع	10^{20}~
Chess	كل أوضاع القطع	10^{43}~
Go	كل أوضاع الحجارة	10^{170}~

ملاحظة: الحجم يحدد **صعوبة** المشكلة!

A: مجموعة الأقواس (Arcs)

تعريف: العمليات/الحركات القانونية التي تنقل من حالة لأخرى

خصائص:

```text
Arc: (state_i, state_j, cost)
```

- **state_i:** الحالة المصدر
- **state_j:** الحالة الهدف
- **cost:** التكلفة (اختياري)

أمثلة:

المشكلة	A (العمليات)
Tic-Tac-Toe	وضع X أو O في خانة فارغة
Puzzle-8	تحريك قطعة مجاورة للفراغ (↑↓←→)
Chess	حركات كل قطعة (حسب القواعد)
خريطة	الطرق بين المدن

S: حالة البداية (Start State)

تعريف: الحالة (أو الحالات) التي نبدأ منها البحث

ملاحظة: عادة S تحتوي على **حالة واحدة**، لكن ممكن أكثر

أمثلة:

المشكلة	S (البداية)
Tic-Tac-Toe	لوحة فارغة
Puzzle-8	الترتيب الابتدائي المعطى
الملاحة	موقعي الحالي (GPS)
لعبة Maze	نقطة الدخول

GD: وصف الهدف (Goal Description)

تعريف: شرط أو وصف للحالة/المسار المطلوب الوصول إليه

نوعان أساسيان:

🔷 أنواع الأهداف 🔥

النوع 1: خاصية الحالة (State Property)

تعريف: نريد الوصول إلى **حالة معينة** أو حالة تحقق شرط معين

```text
Goal = State that satisfies condition C
```

أمثلة:

المشكلة	وصف الهدف
Tic-Tac-Toe	حالة فيها 3 علامات متشابهة في خط
Puzzle-8	الترتيب: `1 2 3 / 4 5 6 / 7 8 _`
Maze	الوصول إلى الخروج
Rubik's Cube	كل وجه بلون واحد

الخاصية:

- نهتم **بالوجهة** فقط
- لا نهتم **بكيفية** الوصول

النوع 2: خاصية المسار (Path Property)

تعريف: نريد مسار **يحقق شرط معين** (ليس مجرد الوصول)

```text
Goal = Path that satisfies condition C
```

أمثلة:

المشكلة	وصف الهدف
TSP	مسار يمر بكل المدن ويعود للبداية
أقصر طريق	المسار ذو **أقل مسافة**
أقل تكلفة	المسار ذو **أقل تكلفة إجمالية**
Knight's Tour	مسار يزور كل خانة مرة واحدة

الخاصية:

- نهتم **بالمسار الكامل**
- ليس فقط الوجهة النهائية

◆ مقارنة شاملة: State vs Path

الخاصية	State Property	Path Property
الهدف	حالة نهائية محددة	مسار يحقق شرط
التقييم	فحص الحالة النهائية	فحص المسار الكامل
المثال	Puzzle-8	TSP
الاهتمام	**أين** وصلنا؟	**كيف** وصلنا؟
التعقيد	عادة أبسط	عادة أعقد

📌 القسم الرابع: مسار الحل (Solution Path)

🔷 التعريف

```text
Solution Path = مسار من S إلى عقدة تحقق GD
```

الصيغة الرسمية:

```text
Path: s₀ → s₁ → s₂ → ... → sₙ
حيث:
- s₀ ∈ S (البداية)
- sₙ يحقق GD (الهدف)
```

🔷 خصائص الحل

الخاصية	الوصف
Existence	هل يوجد حل أصلاً؟
Uniqueness	هل الحل واحد أم متعدد؟
Optimality	هل هو الحل الأفضل؟
Length	طول المسار (عدد الخطوات)
Cost	التكلفة الإجمالية

🔷 أنواع التعقيد

1. خطي (Linear) - O(n)

```text
مثال: البحث في قائمة مرتبة
n = 1000 → 1000 عملية
```

ممتاز ✅

---

2. تربيعي (Quadratic) - O(n²)

```text
مثال: مقارنة كل عنصر بكل عنصر
n = 1000 → 1,000,000 عملية
```

مقبول 🟡

---

3. أسّي (Exponential) - $O(2^n)$

```text
مثال: كل التوليفات الثنائية
n = 20 → 1,048,576 عملية
n = 30 → 1,073,741,824 (مليار!)
```

خطر ❌

---

4. عاملي (Factorial) - $O(n!)$

```text
مثال: كل الترتيبات الممكنة (TSP)
n = 10 → 3,628,800
n = 15 → 1,307,674,368,000 (تريليون!)
n = 20 → 2.4 × 10¹⁸ (كوينتليون!)
```

كارثة ❌❌❌

🔷 جدول مقارنة شامل

التعقيد	n=10	n=20	n=30	التصنيف
$O(n)$	10	20	30	✅ ممتاز
$O(n^2)$	100	400	900	🟡 جيد
$O(n^3)$	1K	8K	27K	🟠 مقبول
$O(2^n)$	1K	1M	1B	❌ سيء
$O(n!)$	3.6M	2.4×10^{18}	2.6×10^{32}	❌ مستحيل

◆ كيف نواجه الانفجار التوافقي؟

الاستراتيجيات:

الطريقة	الوصف	مثال
Pruning	قص الفروع غير الواعدة	Alpha-Beta
Heuristics	استخدام معرفة خاصة بالمجال	A* Search
Approximation	حلول تقريبية جيدة	Greedy TSP
Parallelization	معالجة متوازية	البحث بعدة خيوط
Memoization	حفظ النتائج المحسوبة	Dynamic Programming

State Space Search (البحث في فضاء الحالة)

📌 مقدمة: لماذا نحتاج نظرية البحث؟

◆ الانتقال من المعرفة إلى الحل

الفصل السابق	هذا الفصل
تمثيل المعرفة (Knowledge Representation)	**استخدام المعرفة** (Problem Solving)
كيف نكتب المعلومات؟	كيف نصل للحل؟
Logic, Predicates, Inference	Search Algorithms

◆ الأسئلة الأساسية التي تجيب عليها نظرية البحث

السؤال	ما نريد معرفته
Completeness	هل سنجد الحل إذا كان موجوداً؟
Termination	هل سينتهي البحث أم سندخل في حلقة مفرغة؟
Optimality	هل الحل الذي نجده هو الأمثل؟
Complexity	كم يستغرق من وقت وذاكرة؟

الأداة: نظرية البحث في فضاء الحالة (State Space Search Theory)

📌 القسم الأول: نظرية البيان (Graph Theory)

◆ لماذا Graphs؟

السبب: الرسوم البيانية تكشف **هيكل المشكلة** بشكل أفضل من المنطق

مثال تاريخي: جسور كونيغسبرغ (Königsberg Bridges)

المشكلة:

- مدينة فيها 7 جسور تربط 4 مناطق
- **السؤال:** هل يمكن عبور كل جسر مرة واحدة والعودة لنقطة البداية؟

الحل (ليونارد أويلر - 1736):

1. مثّل المناطق كـ **عقد (Nodes)**
2. مثّل الجسور كـ **أقواس (Edges)**
3. حلّل **درجة كل عقدة** (عدد الأقواس المتصلة بها)
4. **النتيجة:** المشكلة مستحيلة! (بدون محاولة واحدة)

الدرس: التمثيل البياني أقوى من التجربة والخطأ

◆ مكونات الرسم البياني (Graph)

المكون	الوصف	التمثيل
العقد (Nodes/Vertices)	نقاط تمثل الحالات	● ○ ◆
الأقواس (Arcs/Edges)	خطوط تربط العقد	— →

مثال:

```text
        A (أب)
       / \
     B   C (أشقاء)
    /
  D (حفيد لـ A)
```

◆ 6. درجة العقدة (Node Degree)

تعريف: عدد الأقواس المتصلة بالعقدة

النوع	الوصف
In-degree	عدد الأقواس الداخلة
Out-degree	عدد الأقواس الخارجة

مثال:

```text
A → B → C
    ↓
    D
```

العقدة B:

- In-degree = 1 (من A)
- Out-degree = 2 (إلى C و D)

📌 القسم الثالث: فضاء الحالة (State Space)

🔹 التعريف الرسمي

State Space = [N, A, S, GD]

وهو **tuple-4** يتكون من:

الرمز	الاسم	الوصف
N	Nodes	مجموعة **جميع الحالات** الممكنة
A	Arcs	مجموعة **العمليات/الانتقالات** بين الحالات
S	Start state(s)	**الحالة الابتدائية** (مجموعة جزئية من N)
GD	Goal Description	**وصف الهدف** المطلوب الوصول إليه

🔹 شرح كل عنصر

1. N (Nodes - العقد)

تمثل: جميع الحالات الممكنة في المشكلة

أمثلة:

المشكلة	الحالات (N)
Tic-Tac-Toe	كل ترتيبات X و O الممكنة
Puzzle-8	كل ترتيبات الأرقام 1-8 والفراغ
الشطرنج	كل أوضاع القطع الممكنة
خريطة مدن	جميع المدن

2. A (Arcs - الأقواس)

تمثل: العمليات أو الحركات القانونية

أمثلة:

المشكلة	العمليات (A)
Tic-Tac-Toe	وضع X أو O في خانة فارغة
Puzzle-8	تحريك قطعة للفراغ (↑ ↓ ← →)
الشطرنج	حركات القطع القانونية
خريطة مدن	الطرق بين المدن

3. S (Start State - حالة البداية)

تمثل: الحالة الابتدائية للمشكلة

أمثلة:

المشكلة	حالة البداية (S)
Tic-Tac-Toe	لوحة فارغة
Puzzle-8	ترتيب ابتدائي معين
خريطة	المدينة التي نبدأ منها

4. GD (Goal Description - وصف الهدف)

نوعان من الأهداف:

أ. خاصية الحالة (State Property)

تعريف: نريد الوصول لحالة **معينة**

أمثلة:

المشكلة	الهدف
Tic-Tac-Toe	3 علامات متشابهة في خط
Puzzle-8	الترتيب: 1 2 3 / 4 5 6 / 7 8 _
الشطرنج	كش مات للملك

ب. خاصية المسار (Path Property)

تعريف: نريد مسار **يحقق شرط معين**

أمثلة:

المشكلة	الهدف
Traveling Salesperson	مسار يمر بكل المدن ويعود للبداية
أقصر طريق	أقل مسافة بين نقطتين
أقل تكلفة	مسار بأقل تكلفة إجمالية

◆ مسار الحل (Solution Path)

تعريف: مسار يبدأ من **S** وينتهي بعقدة تحقق **GD**

مثال:

```text
START → A → B → GOAL
```

- يبدأ من START (في S)
- ينتهي بـ GOAL (يحقق GD)

📌 القسم الخامس: أنواع الرسوم البيانية

◆ مقارنة شاملة

النوع	الوصف	الدورات	مثال
Undirected Graph	الأقواس بدون اتجاه	ممكنة	خريطة طرق ثنائية الاتجاه
Directed Graph	الأقواس لها اتجاه	ممكنة	الشبكات، علاقات التبعية
DAG	موجه بدون دورات	❌	شجرة القرارات
Tree	مسار واحد بين أي عقدتين	❌	شجرة العائلة
Rooted Tree	شجرة لها جذر محدد	❌	البحث في AI

🔷 متى نستخدم كل نوع؟

المشكلة	النوع المناسب
Tic-Tac-Toe	Directed DAG (لا تراجع، لا دورات)
Puzzle-8	Directed Graph with Cycles
الشطرنج	Directed Graph with Cycles
خريطة مدن	Undirected Graph (أو Directed إذا كانت طرق أحادية)
قواعد بيانات	Directed Graph

📝 أمثلة محلولة

مثال 1: تحديد مكونات State Space

المشكلة: لغز بسيط

```text
لديك 3 أكواب: A, B, C
كوب A ممتلئ بالماء
الهدف: نقل الماء للكوب C
العمليات: صب من كوب لآخر
```

الحل:

```text
N: {(A), (B), (C), (AB), (AC), (BC), (ABC), ...}
   (حالات توزيع الماء)

A: {Pour(A→B), Pour(A→C), Pour(B→A), ...}
   (عمليات الصب)

S: (A)
   (فقط الماء في A)

GD: (C)
    (فقط الماء في C)
```

مسار الحل:

```text
(A) → Pour(A→C) → (C) ✅
```

مثال 2: تحليل رسم بياني

الرسم:

```text
      A
     / \
    B   C
   / \ / \
  D   E   F
```

الأسئلة:

1. هل هو شجرة؟
   - ✅ نعم (لا دورات، مسار واحد بين أي عقدتين)
2. من هو parent لـ E؟
   - B
3. من هم siblings لـ B؟
   - C
4. من هم descendants لـ A؟
   - B, C, D, E, F
5. ما هو out-degree لـ B؟
   - 2 (D و E)

مثال 3: حساب عدد الحالات

المشكلة: Tic-Tac-Toe مصغر (2×2)

```text
الحالات الممكنة نظرياً: 4^3 = 81
(كل خانة: فارغة، X، أو O)

لكن بعضها غير قانوني:
- لا يمكن أن يربحا معاً O و X
- يجب أن يكون عدد X و O متقارب
```

الحالات القانونية:

- بعد التحليل: حوالي 30-35 حالة فقط

🎯 ملخص سريع للحفظ

State Space = [N, A, S, GD]

```text
N  = All possible states (كل الحالات)
A  = Operations (العمليات)
S  = Start state (البداية)
GD = Goal Description (الهدف)
```

أنواع الأهداف:

```text
State Property → حالة معينة (8-Puzzle)
Path Property  → مسار معين (TSP)
```

أنواع Graphs:

```text
Tree        → no cycles, one path
DAG         → directed, no cycles
Directed    → has direction, may have cycles
Undirected  → no direction
```

العلاقات:

```text
Parent     → العقدة السابقة
Child      → العقدة اللاحقة
Siblings   → نفس الأب
Ancestor   → في المسار للأعلى
Descendant → في المسار للأسفل
```

أسئلة النظرية:

```text
Complete? → هل نضمن إيجاد الحل؟
Optimal?  → هل الحل أمثل؟
Finite?   → هل سينتهي؟
Complexity? → التعقيد الزمني والمكاني؟
```

FSM في State Space Search

✅ أساسي - FSM = (S, I, F)

```text
S = مجموعة محدودة من الحالات (States)
I = مجموعة محدودة من المدخلات (Input Alphabet)
F = دالة الانتقال: S × I → S
```

✅ موسّع - FSA = (S, I, F, s_0, A)

```text
s0 = حالة البداية (Start/Initial State)
A = مجموعة حالات القبول (Accept States) حيث A ⊆ S
```

📌 الربط مع State Space Search

المصطلحات المتقابلة:

FSM	State Space Search
State (حالة)	Node (عقدة)
Transition (انتقال)	Arc/Action (قوس/عملية)
Accept State	Goal State (حالة الهدف)
Start State	Initial State (حالة البداية)
Input String	Action Sequence (تسلسل الإجراءات)
FSA accepts	Path reaches goal

الفكرة الجوهرية:

```text
FSM = نموذج خاص من State Space حيث:
- الحالات محدودة (Finite)
- الانتقالات محددة بوضوح (Deterministic/Non-deterministic)
- الهدف: قبول/رفض سلسلة (Pattern Recognition)
```

📌 جدول المقارنة الحاسم (يخرج منه 60% من الأسئلة)

الخاصية	Deterministic FSM	Non-Deterministic FSM
التعريف الرياضي	F: S × I → S (دالة)	F: S × I → P(S) (علاقة/مجموعة)
عدد الانتقالات	واحد فقط لكل (s, i)	صفر أو أكثر لكل (s, i)
التنفيذ	مباشر وسهل	يحتاج backtracking
التعقيد الزمني	O(n) حيث n = طول السلسلة	O(2^n) في أسوأ حالة
الذاكرة	O(1) - حالة واحدة فقط	O(k) - حيث k عدد الحالات المحتملة
القوة	أضعف (لكن كافٍ)	أقوى (لكن يمكن تحويله)
في الامتحان	✅ الأكثر شيوعاً	🟡 قد يُسأل عن التعريف
مثال	آلة قبول الأرقام الزوجية	آلة بها ε-transitions

النظرية الحاسمة:

```text
كل NFSM يمكن تحويله إلى DFSM مكافئ (Power Set Construction)
لكن قد يزداد عدد الحالات أُسّياً (n^2)
```

📌 كيفية تتبع (Tracing) سلسلة باستخدام الجدول - منهجية الامتحان

الخطوات الثلاث (احفظها!) 🔥

الخطوة 1: حدد الحالة الابتدائية

```text
Current State = s₀
```

الخطوة 2: لكل رمز في السلسلة، طبّق:

```text
Next State = Table[Current State][Input Symbol]
Current State = Next State
```

الخطوة 3: افحص الحالة النهائية

```text
إذا Current State ∈ A → مقبول ✅
إذا Current State ∉ A → مرفوض ❌
```

مثال تطبيقي (نمط الامتحان):

الجدول المعطى:

State	'a'	'b'	'c'
S_0	S_1	S_0	S_0
S_1	S_1	S_2	S_0
S_2	S_1	S_0	S_3
S_3	S_1	S_0	S_0

المعلومات:

```text
s₀ = Start State
A = {s₃}
```

السؤال: هل السلسلة "abc" مقبولة؟

الحل المنهجي:

```text
Step 0: Current = s₀

Input 'a':
  Table[s₀]['a'] = s₁
  Current = s₁ ✅

Input 'b':
  Table[s₁]['b'] = s₂
  Current = s₂ ✅

Input 'c':
  Table[s₂]['c'] = s₃
  Current = s₃ ✅

Final Check:
  s₃ ∈ {s₃}? YES ✅

الإجابة: مقبول ✅
```

السؤال: هل السلسلة "abbc" مقبولة؟

الحل:

```text
Current = s₀

'a' → s₁
'b' → s₂
'b' → Table[s₂]['b'] = s₀ ❌ (عدنا للبداية!)
'c' → Table[s₀]['c'] = s₀

Final: s₀ ∉ {s₃}

❌ الإجابة: مرفوض
```

📌 الـ 10 حقائق الامتحانية (لا يخرج عنها سؤال)

✅ 1. التعريف الثلاثي

```text
FSM = (S, I, F) هو الحد الأدنى
FSA يضيف s₀ و A
Moore Machine = FSA
```

✅ 2. دالة الانتقال

```text
في DFSM: F(s, i) = حالة واحدة فقط
في NFSM: F(s, i) = مجموعة من الحالات (قد تكون فارغة)
```

✅ 3. شرط القبول

```text
String accepted ⟺ Final State ∈ A
ليس المهم المسار، المهم أين انتهينا
```

✅ 4. حالة البداية

```text
في FSA: s₀ واحدة فقط ومحددة
في FSM العام: قد لا تكون محددة
```

✅ 5. عدد الحالات

```text
FINITE = محدود (وهذا سبب التسمية)
إذا غير محدود → ليس FSM (مثل Turing Machine)
```

✅ 6. قراءة جدول الانتقال

```text
الصف = الحالة الحالية
العمود = المدخل
الخلية = الحالة التالية
```

✅ 7. الحالات الخاصة

```text
Start State: → ●
Accept State: ◎ (دائرة مزدوجة)
Trap State: حالة non-accepting لا يمكن الخروج منها
```

✅ 8. التحويل بين التمثيلات

```text
State Diagram ↔ Transition Table
كلاهما مكافئ تماماً
```

✅ 9. DFSM vs NFSM

```text
NFSM أسهل في التصميم
DFSM أسهل في التنفيذ
كل NFSM له DFSM مكافئ
```

✅ 10. التطبيقات في AI

```text
- Pattern Matching (البحث عن أنماط)
- Lexical Analysis (تحليل الرموز)
- Protocol Design (تصميم البروتوكولات)
- Game AI (سلوك الشخصيات: Idle, Attack, Flee)
```

📌 نقاط ذكية للـ MCQ (Bullet Points)

عن التعريف:

- ✅ FSM = 3-tuple: (S, I, F)
- ✅ FSA = 5-tuple: (S, I, F, s_0, A)
- ✅ S و I **محدودة** (finite)
- ✅ F هي **دالة** في DFSM، **علاقة** في NFSM
- ❌ FSM ≠ Turing Machine (TM غير محدود)

عن الانتقالات:

- ✅ DFSM: انتقال واحد فقط لكل (state, input)
- ✅ NFSM: صفر أو أكثر من انتقال
- ✅ ε-transition: انتقال بدون قراءة رمز (في NFSM فقط)
- ✅ Self-loop: الحالة تنتقل لنفسها

عن القبول:

- ✅ يتم قبول السلسلة إذا وفقط إذا: الحالة النهائية ∈ A
- ✅ قد يكون A = ∅ (لا قبول أبداً)
- ✅ قد يكون A = S (قبول دائماً)
- ❌ الوصول لحالة accept في المنتصف لا يكفي

عن الجداول:

- ✅ كل خلية تحتوي **حالة واحدة** في DFSM
- ✅ كل خلية تحتوي **مجموعة حالات** في NFSM
- ✅ خلية فارغة في NFSM = رفض (لا انتقال)
- ✅ عدد الصفوف = |S|، عدد الأعمدة = |I|

عن الرسم البياني:

- ✅ العقد = الحالات
- ✅ الأقواس = الانتقالات
- ✅ التسمية على القوس = رمز المدخل
- ✅ → من الفراغ = start state
- ✅ ◎ دائرة مزدوجة = accept state

📝أسئلة MCQ امتحانية مركزة (15 سؤال)

Q1: In DFSM, the transition function F is:

- A) A relation: S x I -> P(S)
- **B) A function: S x I -> S** ✅
- C) A set: S x I
- D) Undefined

- **شرح:** في DFSM ، لكل زوج (state, input) يوجد حالة واحدة ناتجة فقط = دالة.

Q2: FSA differs from FSM by having:

- A) More states
- **B) Designated start and accept states** ✅
- C) Probabilistic transitions
- D) No transitions

- **شرح:** FSA = FSM + s0 + A (تحديد البداية والنهاية.)

Q3: A string is accepted by FSA if:

- A) It passes through an accept state
- **B) It ends in an accept state** ✅
- C) Start state is accept state
- D) All states are accept states

- **شرح:** المهم أين ننتهي عند آخر حرف في السلسلة، وليس المسار.

Q4: In NFSM, F(s, i) can be:

- A) One state only
- B) Empty set
- C) Multiple states
- **D) Any of the above** ✅

- **شرح:** NFSM يسمح بـ: لا انتقال (مجموعة خالية)، انتقال واحد، أو عدة انتقالات لنفس المدخل.

Q5: The "finite" in FSM refers to:

- A) Input string length
- **B) Number of states** ✅
- C) Transition time
- D) Accept states only

- **شرح:** S هي مجموعة محدودة.(Finite Set)

Q6: Given Table[s0]['a'] = s1, this means:

- A) s0 is start state
- **B) From s0, input 'a' leads to s1** ✅
- C) s1 accepts 'a'
- D) s0 and s1 are equal

Q7: A trap state is:

- A) An accept state
- B) The start state
- **C) A non-accept state with only self-loops** ✅
- **شرح :**حالة غير مقبولة، بمجرد دخولها تكرر نفسها ولا يمكن الخروج منها.

Q8: In FSM graph, double circle means:
- A) Start state
- **B) Accept state** ✅
- C) Error state
- D) Intermediate state

Q9: epsilon-transition (ε) means:

- A) No transition
- B) Error transition
- **C) Transition without reading input** ✅
- D) Final transition
- **شرح :**انتقال تلقائي دون استهلاك مدخلات، موجود في NFSM فقط.

Q10: Every NFSM can be converted to:

- A) Turing Machine
- **B) Equivalent DFSM** ✅
- C) Push-Down Automaton
- D) Context-Free Grammar

Q11: For string "abc" with FSA having A={s3}, if we end at s0:

- A) Accepted
- **B) Rejected** ✅
- C) Undefined
- D) Partially accepted
- **شرح :**بما أن s0 ليست من ضمن مجموعة حالات القبول(A) ، فالسلسلة مرفوضة.

Q12: FSM with S={s0, s1} and I={0, 1} has ___ entries in transition table:

- A) 2
- **B) 4** ✅
- C) 8
- D) Infinite
- **شرح :**عدد الحالات (2) × عدد المدخلات (2) = 4 خلايا.

Q13: In State Space Search, FSM's "Accept State" corresponds to:
- A) Start node
- **B) Goal state** ✅
- C) Any node
- D) Dead end

Q14: Moore Machine is another name for:

- A) Turing Machine
- **B) Finite State Acceptor** ✅
- C) Push-Down Automaton
- D) Linear Bounded Automaton

Q15: In transition table, rows represent:

- A) Inputs
- **B) Current states** ✅
- C) Next states
- D) Accept states

🎯 ملخص نهائي للحفظ السريع

الصيغة:

```text
FSM = (S, I, F)
FSA = (S, I, F, s₀, A)
```

الفرق الحاسم:

```text
DFSM: F(s,i) = one state
NFSM: F(s,i) = set of states (0 or more)
```

شرط القبول:

```text
Accepted ⟺ Final State ∈ A
```

الربط مع AI:

```text
State → Node
Transition → Arc/Action
Accept State → Goal State
Start State → Initial State
```

قراءة الجدول:

```text
Table[current_state][input] = next_state
```

الحقيقة الذهبية:

```text
كل NFSM ← يمكن تحويله ← DFSM مكافئ
```

State Space Representation

📌 التعريف الرياضي (Tuple-4) - احفظه!

```text
State Space = [N, A, S, GD]
```

الرمز	المعنى	الوصف بجملة واحدة
N	Nodes	مجموعة **جميع الحالات الممكنة** في المشكلة
A	Arcs	مجموعة **العمليات/الخطوات** التي تنقل بين الحالات
S	Start State(s)	مجموعة جزئية من N تمثل **نقطة البداية** (∅ ≠ S ⊆ N, S)
GD	Goal Description	**وصف الحالة أو المسار** المطلوب الوصول إليه

Solution Path: مسار من عقدة في S إلى عقدة تحقق GD

📌 أنواع الأهداف (Goal Description Types)

الخاصية	Type 1: State Property	Type 2: Path Property
التعريف	هدف يعتمد على **خصائص الحالة النهائية**	هدف يعتمد على **خصائص المسار الكامل**
الاهتمام	**أين** وصلنا؟	**كيف** وصلنا؟
التقييم	فحص الحالة النهائية فقط	فحص المسار من البداية للنهاية
المثال 1	Puzzle-8: الوصول للترتيب المطلوب	TSP: أقل مسافة إجمالية
المثال 2	Tic-Tac-Toe: 3 علامات متشابهة	Shortest Path: أقصر طريق
الصيغة	Goal = Specific State Configuration	Goal = Path that minimizes/maximizes cost
في الامتحان	✅ الأكثر شيوعاً	🟡 يُسأل عنه في TSP

📌 جدول الأمثلة التطبيقية (يخرج منه 40% من الأسئلة)

المشكلة	N (الحالات)	نوع الرسم	Cycles?	نوع GD	الخصائص الفريدة	التعقيد
Tic-Tac-Toe	كل توزيعات X و O	DAG (Directed Acyclic Graph)	❌ لا	State Property	- ليس شجرة (يمكن الوصول لحالة بمسارات مختلفة) - لا يمكن التراجع عن حركة - متصل (Connected)	~5,478 حالة قانونية ✅ قابل للحل
8-Puzzle	ترتيبات 1-8 + فراغ	Directed Graph	✅ نعم	State Property	- **فضاء منفصل** (2 مجموعات منفصلة) - نصف الحالات غير قابلة للوصول - خطر الحلقات المفرغة	181,440 = 9!/2 حالة 🟡 يحتاج خوارزميات ذكية
TSP	تسلسل المدن المزارة	Complete Graph (Weighted)	✅ نعم	Path Property	- **انفجار توافقي** Combinatorial Explosion - الهدف: أقل مسافة/تكلفة - يجب زيارة كل مدينة مرة واحدة	(n-1)!/2 ❌ 20 مدينة = مستحيل exhaustive search

📌 الـ 7 حقائق التقنية الذكية (لا يخرج عنها سؤال)

✅ 1. DAG vs Cyclic Graph

```text
DAG (Directed Acyclic Graph):
  - لا توجد دورات (no cycles)
  - لا يمكن العودة لحالة سابقة
  - مثال: Tic-Tac-Toe

Cyclic Graph:
  - توجد دورات (cycles exist)
  - يمكن العودة لحالة سابقة
  - خطر: حلقات مفرغة (infinite loops)
  - مثال: 8-Puzzle, TSP
```

✅ 2. الانفصال في Puzzle-8

```text
فضاء الحالة منقسم لمجموعتين منفصلتين:
- مجموعة A: 181,440 حالة (قابلة للحل من بعضها)
- مجموعة B: 181,440 حالة (قابلة للحل من بعضها)
- لا يوجد مسار من A إلى B بدون كسر اللعبة

السبب: Parity (التكافؤ الرياضي)
```

✅ 3. الانفجار التوافقي (Combinatorial Explosion)

```text
TSP Complexity = (n-1)! / 2

أمثلة:
n = 10 cities → 181,440 paths
n = 20 cities → 6 × 10¹⁶ paths (exhaustive search مستحيل!)
n = 50 cities → أكبر من عمر الكون

الحل: Heuristic Search (بحث ذكي)
```

```text
الأقواس تُعرَّف بحركة الفراغ (blank) وليس الأرقام
السبب: تبسيط القواعد (4 حركات فقط: ↑ ↓ ← →)

بدلاً من: تحريك رقم 1, 2, 3, ... 8
نستخدم: تحريك الفراغ (أبسط وأوضح)
```

✅ 5. الفرق بين Graph و Tree

```text
Tree:
  - مسار واحد فقط بين أي عقدتين
  - لا دورات
  - كل عقدة (عدا الجذر) لها أب واحد

Graph (ليس Tree):
  - قد توجد مسارات متعددة
  - قد توجد دورات
  - العقدة قد يكون لها عدة آباء

Tic-Tac-Toe = Graph (not Tree)
لأن: نفس الحالة يمكن الوصول إليها بترتيبات مختلفة
```

✅ 6. الأوزان (Weights) في الأقواس

```text
Unweighted Arcs:
- كل الخطوات متساوية
- مثال: Puzzle-8 (كل حركة = 1)

Weighted Arcs:
- كل قوس له تكلفة/مسافة
- مثال: TSP (مسافات مختلفة بين المدن)
- الهدف: تقليل المجموع الكلي
```

✅ 7. حالة البداية (Start State)

```text
S ⊆ N (مجموعة جزئية)
S ≠ ∅ (غير فارغة)

عادةً: S تحتوي على حالة واحدة فقط
لكن: قد تحتوي على أكثر (في بعض المشكلات)

مثال:
- Puzzle: S-8 = {الترتيب الابتدائي المعطى}
- Tic-Tac-Toe: S = {لوحة فارغة}
```

📌 نقاط ذكية للـ MCQ

عن التمثيل:

- ✅ State Space = رسم بياني (Graph)
- ✅ Nodes = States (الحالات)
- ✅ Arcs = Operations/Actions (العمليات)
- ✅ التعريف الرسمي = **4-tuple: [N, A, S, GD]**

عن الأهداف:

- ✅ Type 1 (State): نهتم بالحالة النهائية
- ✅ Type 2 (Path): نهتم بالمسار الكامل
- ✅ Puzzle = Type 1-8
- ✅ TSP = Type 2

عن **Tic-Tac-Toe**:

- ✅ DAG (لا دورات)
- ✅ ليس Tree (مسارات متعددة لنفس الحالة)
- ✅ لا يمكن التراجع
- ✅ متصل (Connected)

عن **Puzzle-8**:

- ✅ يحتوي دورات (خطر الحلقات المفرغة)
- ✅ فضاء منفصل (Disconnected) - مجموعتان
- ✅ نصف الحالات غير قابلة للحل
- ✅ الأقواس = حركة الفراغ (4 اتجاهات)

عن **TSP**:

- ✅ الهدف = Path Property (أقل تكلفة/مسافة)
- ✅ Weighted Graph (أقواس بأوزان)
- ✅ Combinatorial Explosion: (n-1)!/2
- ✅ Exhaustive search مستحيل لـ n>20

عن الدورات **(Cycles)**:

- ✅ خطر: infinite loops
- ✅ الحل: تتبع الحالات المزارة (Visited Set)
- ✅ DAG = لا دورات (safe)
- ✅ Cyclic Graph = يحتاج معالجة خاصة

📝أسئلة MCQ امتحانية (20 سؤال)

Q1: State Space is formally defined as:

- A) A 3-tuple (S, I, F)
- **B) A 4-tuple [N, A, S, GD]** ✅
- C) A 5-tuple (S, I, F, s0, A)
- D) A graph only

Q2: In [N, A, S, GD], N represents:

- A) Number of nodes
- **B) Set of all possible states** ✅
- C) Next state
- D) Node transitions

Q3: GD stands for:

- A) Graph Description
- **B) Goal Description** ✅
- C) General Direction
- D) Graph Diagram

Q4: S (Start State) is:

- A) A single state
- **B) A non-empty subset of N** ✅
- C) The goal state
- D) Any state

Q5: Type 1 Goal Description depends on:

- A) Path cost
- **B) State properties** ✅
- C) Number of steps
- D) Search algorithm

Q6: Type 2 Goal Description depends on:

- A) Final state only
- **B) Path properties (cost, length, etc.)** ✅
- C) Start state
- D) Number of nodes

Q7: 8-Puzzle goal is:

- A) Path Property
- **B) State Property** ✅
- C) Both

- D) Neither

Q8: TSP (Traveling Salesman Problem) goal is:

- A) State Property
- **B) Path Property** ✅
- C) Both
- D) Neither

Q9: Tic-Tac-Toe state space is:

- A) A tree
- **B) A DAG (Directed Acyclic Graph)** ✅
- C) Cyclic graph
- D) Disconnected

Q10: DAG means:

- A) Direct Acyclic Graph
- **B) Directed Acyclic Graph** ✅
- C) Double Acyclic Graph
- D) Divided Acyclic Graph

Q11: 8-Puzzle state space is:

- A) A tree
- B) A DAG
- **C) Disconnected (2 separate components)** ✅
- D) Complete graph

Q12: How many states are unreachable in 8-Puzzle from any starting configuration?

- A) None
- B) 25%
- **C) 50% (half of all possible states)** ✅
- D) 75%

Q13: In 8-Puzzle, arcs are defined by:

- A) Moving numbered tiles
- **B) Moving the blank space** ✅
- C) Swapping any two tiles
- D) Random moves

Q14: Combinatorial Explosion occurs in:

- A) 8-Puzzle
- B) Tic-Tac-Toe
- C) TSP
- **D) All of the above** ✅

Q15: For TSP with n cities, number of possible tours is:

- A) n!
- **B) (n-1)! / 2** ✅
- C) 2^n
- D) n^2

Q16: Weighted arcs means:

- A) Heavy edges
- **B) Arcs have associated costs/distances** ✅
- C) Multiple arcs
- D) No arcs

Q17: A graph with cycles requires:

- A) No special handling
- **B) Tracking visited states to avoid loops** ✅
- C) Deleting cycles
- D) Reversing arcs

Q18: Tic-Tac-Toe is NOT a tree because:

- A) It has cycles
- **B) Same state reachable via different paths** ✅
- C) It's disconnected
- D) It has no root

Q19: Solution Path is:

- A) Any path in the graph
- **B) Path from S to a state satisfying GD** ✅
- C) The shortest path
- D) Path with maximum cost

Q20: In TSP, the goal requires:

- A) Visiting one city
- **B) Visiting all cities and returning with minimum cost** ✅
- C) Any path
- D) Shortest path between two cities

🎯 ملخص نهائي (احفظه كله!)

الصيغة الأساسية:

أنواع الأهداف:

```text
Type 1 (State Property):
- Puzzle-8 ✅
- Tic-Tac-Toe ✅

Type 2 (Path Property):
- TSP ✅
- Shortest Path ✅
```

الأمثلة الثلاثة (الجدول):

```text
Tic-Tac-Toe:  DAG, No Cycles, ~5K states
8-Puzzle:     Cyclic, Disconnected, 181K states
TSP:          Weighted, Combinatorial Explosion
```

الحقائق السبع:

```text
1. DAG = No Cycles (Tic-Tac-Toe)
2. 8-Puzzle = 50% states unreachable
3. TSP = (n-1)!/2 complexity
4. 8-Puzzle arcs = blank movement
5. Graph ≠ Tree (multiple paths)
6. Weighted arcs = costs/distances
7. S ⊆ N and S ≠ ∅
```

للامتحان:

```text
✅ احفظ [N, A, S, GD]
✅ Type 1 vs Type 2 Goals
✅ DAG (Tic-Tac-Toe) vs Cyclic (8-Puzzle)
✅ Disconnected space (8-Puzzle)
✅ Combinatorial Explosion (TSP)
```

Search Strategies

📌 الجزء الأول: اتجاه البحث (Search Direction)

جدول المقارنة الأساسي

الخاصية	Data-Driven	Goal-Driven
المصطلح البديل	Forward Chaining (التسلسل الأمامي)	Backward Chaining (التسلسل الخلفي)
نقطة البداية	المعطيات/الحقائق المتوفرة (Facts)	الهدف النهائي المطلوب (Goal)
الاتجاه	من البداية → الهدف	من الهدف ← البداية
الآلية	تطبيق القواعد على البيانات لتوليد حقائق جديدة	تحليل الهدف لأهداف فرعية (Subgoals)
يُستخدم عندما	- البيانات كلها متوفرة في البداية - الهدف غير واضح - عدد الأهداف المحتملة كبير	- الهدف واضح ومحدد - البيانات غير متوفرة (نحتاج جمعها) - تفرع البيانات كبير جداً
مثال تطبيقي	- التفسير الجيولوجي - تحليل البيانات الموجودة	- إثبات نظرية رياضية - التشخيص الطبي (نوجه الفحوصات)
في المنهج	✅ Chapter 3, Section 3.4.1	✅ Chapter 3, Section 3.4.2

📌 الجزء الثاني: آلية التراجع (Backtracking Mechanism)

القوائم الثلاث (Luger's Implementation) 🔥

القائمة	الوظيفة بجملة واحدة
SL (State List)	تحتفظ بالحالات في **المسار الحالي** (الحل المحتمل الذي نختبره)
NSL (New State List)	تحتوي على الحالات **المولّدة والمنتظرة للفحص** (تعمل كـ Open List)
DE (Dead Ends)	تسجل الحالات **التي فُحصت ولم تؤدِ لحل** (لتجنب إعادة زيارتها)

الآلية:

```text
1. ابدأ من حالة في NSL
2. أضفها لـ SL (المسار الحالي)
3. ولّد أبناءها وأضفهم لـ NSL
4. إذا وصلت لطريق مسدود:
   - انقل الحالة من SL إلى DE
   - تراجع (Backtrack) للحالة السابقة في SL
5. كرر حتى تجد الهدف أو تفرغ NSL
```

📌 الجزء الثالث: ترتيب البحث + هياكل البيانات

الربط الجوهري (يخرج منه 50% من الأسئلة)

الاستراتيجية	هيكل البيانات	آلية العمل	الترتيب
BFS	Queue (FIFO)	First In, First Out	يفحص المستوى n بالكامل قبل n+1
DFS	Stack (LIFO)	Last In, First Out	يغوص عميقاً قبل فحص الأشقاء
Iterative Deepening	Stack (LIFO) + Depth Bound	DFS محدود العمق، يُعاد مع زيادة الحد	يجمع ميزات BFS و DFS

التنفيذ التقني

BFS (Breadth-First Search):

```text
Open List = Queue (FIFO)

الإضافة: أضف العقد الجديدة في النهاية (Enqueue)
السحب: اسحب من البداية (Dequeue)

النتيجة:
... → Level 0 → Level 1 → Level 2
```

:DFS (Depth-First Search)

```text
Open List = Stack (LIFO)

الإضافة: أضف العقد الجديدة في البداية (Push)
السحب: اسحب من البداية (Pop)

النتيجة:
يذهب عميقاً لأقصى حد قبل التفرع
```

📌 الجزء الرابع: جدول المقارنة التقني الشامل

الخاصية	BFS	DFS	Iterative Deepening
هيكل البيانات	Queue (FIFO)	Stack (LIFO)	Stack + Depth Bound
أقصر مسار	✅ **يضمن** إيجاده	❌ لا يضمن	✅ **يضمن** إيجاده
الذاكرة	❌ **O(b^d)** أسّي	✅ **O(b×d)** خطي	✅ **O(b×d)** خطي
الاكتمال	✅ Complete (إذا b محدود)	❌ قد يدخل حلقة لا نهائية	✅ Complete
الوقت	O(b^d)	O(b^d)	O(b^d) لكن أبطأ قليلاً
المشكلة الرئيسية	🔥 Memory Explosion	**قد يضيع في مسار عميق**	إعادة البحث (لكن التكلفة مقبولة)
الأفضل في	- فضاءات ضحلة - نريد أقصر حل	- فضاءات عميقة جداً - الذاكرة محدودة	- **الحل الأمثل عملياً** - يجمع ميزات الاثنين
في الامتحان	✅ الأكثر شيوعاً	✅ شائع	🟡 قد يُسأل

التعقيد التفصيلي (مهم للـ MCQ)

الرموز:

- **b** = Branching Factor (عدد الأبناء لكل عقدة)
- **d** = Depth (عمق الحل)

المقياس	BFS	DFS	Iterative Deepening
Time Complexity	O(b^d)	O(b^m) حيث m = max depth	O(b^d)
Space Complexity	**O(b^d)** ← المشكلة!	**O(b×d)** ← الميزة!	**O(b×d)** ← ميزة!

مثال رقمي (انفجار الذاكرة):

```text
b = 10 (كل عقدة لها 10 أبناء)
d = 6 (عمق الحل)

BFS Memory: 10^6 = 1,000,000 عقدة في الذاكرة ❌
DFS Memory: 10 × 6 = 60 عقدة في الذاكرة ✅

لذا: BFS مستحيل في الشطرنج (b≈35, d≈100)
```

📌 الجزء الخامس: المصطلحات التقنية الحاسمة

✅ 1. Forward Chaining

```text
Data-Driven Search =
= البدء من الحقائق وتطبيق القواعد للأمام
= مثال: "لدي أعراض X و Y، ما التشخيص؟"
```

✅ 2. Backward Chaining

```text
Goal-Driven Search =
= البدء من الهدف والعمل للخلف
= مثال: "لإثبات النظرية Z، ما الذي أحتاجه؟"
```

✅ 3. Combinatorial Explosion

```text
= نمو فضاء البحث بشكل أسّي (Exponential)
= السبب الرئيسي لمشكلة BFS
= مثال: b=10, d=10 → 10^10 = 10 مليار حالة!
```

✅ 4. Backtracking

```text
= التراجع عند الوصول لطريق مسدود
= العودة لأقرب نقطة تفرع لم تُفحص
= أساسي في DFS
```

✅ 5. FIFO (First In, First Out)

```text
= هيكل Queue (الطابور)
= يُستخدم في BFS
= مثال: الصف في البنك
```

✅ 6. LIFO (Last In, First Out)

```text
= هيكل Stack (المكدس)
= يُستخدم في DFS
= مثال: صندوق الكتب (آخر ما تضع، أول ما تخرج)
```

✅ 7. Depth Bound

```text
= حد أقصى للعمق في Iterative Deepening
= يمنع DFS من الضياع
= يزداد تدريجياً: 1, 2, 3, 4, ...
```

✅ 8. Complete Algorithm

```text
= يضمن إيجاد الحل إذا كان موجوداً
= BFS: Complete ✅
= DFS: Not Complete ❌
= Iterative Deepening: Complete ✅
```

✅ 9. Optimal Algorithm

```text
= يضمن إيجاد أقصر/أفضل حل
= BFS: Optimal ✅ (إذا كل الخطوات متساوية)
= DFS: Not Optimal ❌
= Iterative Deepening: Optimal ✅
```

📌 الجزء السادس: Iterative Deepening (الحل الهجين)

المشكلة التي يحلها:

```text
BFS: ذاكرة ضخمة ❌
DFS: قد يضيع ❌
الحل: Iterative Deepening ✅
```

الآلية:

```text
Iteration 1: DFS بحد عمق = 1
Iteration 2: DFS بحد عمق = 2
Iteration 3: DFS بحد عمق = 3
...
حتى نجد الحل
```

المميزات:

- ✅ يضمن أقصر مسار (مثل BFS)
- ✅ ذاكرة قليلة (مثل DFS)
- ✅ Complete
- ✅ Optimal

العيوب:

- 🟡 إعادة فحص العقد (لكن التكلفة مقبولة!)

لماذا التكلفة مقبولة؟

```text
معظم العقد في المستويات الأخيرة:
Level 0: 1 عقدة
Level 1: b عقدة
Level 2: b² عقدة
Level d: b^d عقدة ← معظم العمل هنا!

لذا: إعادة فحص المستويات العليا لا يكلف كثيراً
```

📝أسئلة MCQ امتحانية (25 سؤال)

Q1: Data-Driven search is also known as:

- A) Backward Chaining
- **B) Forward Chaining** ✅
- C) Depth-First
- D) Breadth-First
- **شرح :**البحث الموجه بالبيانات يبدأ من المعطيات ليصل للهدف (سحب أمامي).

Q2: Goal-Driven search is also known as:

- A) Forward Chaining
- **B) Backward Chaining** ✅
- C) Breadth-First
- D) Iterative Deepening
- **شرح :**يبدأ من الهدف ويعود للخلف للبحث عن البيانات التي تدعمه.

Q3: BFS uses which data structure?

- A) Stack
- **B) Queue (FIFO)** ✅
- C) Heap
- D) Tree
- **شرح :**خوارزمية BFS تعتمد على الطابور (الأولوية لمن دخل أولاً).

Q4: DFS uses which data structure?

- A) Queue
- **B) Stack (LIFO)** ✅
- C) Array
- D) Hash Table
- **شرح :**خوارزمية DFS تعتمد على المكدس (الأولوية لآخر عنصر مضاف).

Q5: Which guarantees finding the shortest path?

- A) DFS
- **B) BFS** ✅
- C) Both
- D) Neither

Q6: Which has better memory efficiency?

- A) BFS
- **B) DFS** ✅
- C) Equal
- D) Depends on graph
- **شرح :**الـ DFS يحتاج فقط لتخزين المسار الحالي، بينما BFS يخزن كل الحالات في المستوى الحالي.

Q7: The SL (State List) contains:

- A) All generated states
- **B) States in current path** ✅
- C) Dead ends

Q8: The NSL (New State List) contains:

- A) Visited states
- **B) States waiting to be examined** ✅
- C) Dead ends only
- D) Final solutions

Q9: The DE (Dead Ends) list contains:

- A) Goal states
- B) Current path
- **C) States that led nowhere** ✅

Q10: BFS memory complexity is:

- A) O(b x d)
- **B) O(b^d)** ✅
- C) O(d)
- D) O(b)
- **شرح:** تعقيد الذاكرة في BFS ينمو أسياً مع العمق.

Q11: DFS memory complexity is:

- A) O(b^d)
- **B) O(b x d)** ✅
- C) O(d^2)
- D) O(b^2)

Q12: Iterative Deepening combines advantages of:

- A) Forward and Backward
- **B) BFS and DFS** ✅
- C) SL and NSL
- D) Data and Goal driven
- **شرح:** يأخذ كفاءة الذاكرة من DFS وضمان الوصول للهدف (Completeness) من BFS.

Q13: FIFO stands for:

- A) Fast In, Fast Out
- **B) First In, First Out** ✅
- C) Final In, Final Out
- D) Fixed In, Fixed Out

Q14: LIFO stands for:

- A) Long In, Fast Out
- **B) Last In, First Out** ✅
- C) Linear In, Final Out
- D) Limited In, Fixed Out

Q15: Combinatorial Explosion refers to:
- A) Linear growth
- **B) Exponential growth of search space** ✅
- C) Constant complexity
- D) Memory reduction

Q16: Which is complete (guarantees finding solution)?
- A) DFS only
- B) BFS only
- **C) BFS and Iterative Deepening** ✅

Q17: Backtracking is most associated with:
- A) BFS
- **B) DFS** ✅
- C) Forward Chaining

Q18: Goal-Driven is preferred when:

- A) All data available
- **B) Goal is clear, data must be acquired** ✅
- C) Branching factor is low
- D) Memory is unlimited

Q19: Data-Driven is preferred when:
- A) Goal is clear
- **B) All data available, goal unclear** ✅
- C) Deep search needed

Q20: Iterative Deepening memory complexity is:
- A) O(b^d)
- **B) O(b x d)** ✅
- C) O(d^2)

Q21: A complete algorithm:
- A) Finds optimal solution
- **B) Finds solution if exists** ✅
- C) Uses minimum memory

Q22: An optimal algorithm:
- A) Runs fastest
- B) Uses least memory
- **C) Finds best solution** ✅

Q23: In medical diagnosis, which strategy is better?

- A) Data-Driven
- **B) Goal-Driven (guides which tests to order)** ✅
- C) BFS

Q24: Queue operations in BFS:

- A) Push/Pop
- **B) Enqueue/Dequeue** ✅
- C) Insert/Delete

🎯 ملخص نهائي (احفظه!)

الاتجاه:

```text
:Data-Driven (Forward Chaining)
  - ابدأ من البيانات
  - استخدمه: البيانات متوفرة، الهدف غامض

:Goal-Driven (Backward Chaining)
  - ابدأ من الهدف
  - استخدمه: الهدف واضح، البيانات تُجمع
```

هياكل البيانات:

```text
BFS → Queue (FIFO)
DFS → Stack (LIFO)
```

القوائم:

```text
SL  = المسار الحالي
NSL = المنتظرة للفحص
DE  = الطرق المسدودة
```

المقارنة:

```text
BFS: أقصر مسار ✅, ذاكرة ضخمة ❌
DFS: ذاكرة قليلة ✅, قد يضيع ❌
Iterative Deepening: الأفضل عملياً ✅✅
```

المصطلحات:

```text
```

Logical Reasoning in State Space

📌 الجزء الأول: جدول الربط المنطقي

المقابلة بين **Logic** و **State Space**

عنصر المنطق	عنصر فضاء الحالة	الوظيفة
حقيقة/عبارة منطقية (Fact/Proposition)	**عقدة** (Node)	تمثل معرفة أو حالة منطقية
قاعدة استنتاج (Inference Rule)	**قوس** (Arc)	ينقل من مقدمة إلى نتيجة
مودوس بونينز Modus Ponens	**عملية الانتقال** (Transition)	تطبيق القاعدة = عبور القوس
برهان منطقي (Logical Proof)	**مسار حل** (Solution Path)	سلسلة من التطبيقات المتتالية
الحقائق المعروفة (Known Facts)	**حالة البداية** (Start State)	نقطة الانطلاق (S)
العبارة المطلوب إثباتها (Goal)	**وصف الهدف** (Goal Description)	ما نريد الوصول إليه (GD)
قاعدة AND $(q \wedge r \rightarrow p)$	**قوس فائق** (Hyperarc)	يربط مجموعة عقد بعقدة واحدة
قاعدة OR $(q \vee r \rightarrow p)$	**أقواس منفصلة**	طرق بديلة للوصول للهدف

الفكرة الجوهرية:

```text
الاستدلال المنطقي = البحث في رسم بياني
إثبات نظرية = إيجاد مسار من الحقائق إلى الهدف
```

📌 الجزء الثاني: And/Or Graphs - الفرق الجوهري

متى نستخدم كلاً منهما؟

الخاصية	OR Relation	AND Relation
الصيغة المنطقية	$q \vee r \rightarrow p$	$q \wedge r \rightarrow p$
المعنى	إثبات **أحدهما** كافٍ	إثبات **كليهما** ضروري
التمثيل البياني	أقواس منفصلة (قوسان مستقلان)	أقواس مربوطة بخط منحني
الحل	نجاح **أي** مسار يكفي	يجب نجاح **كل** المسارات
مثال رياضي	طريقتان بديلتان للتكامل	تحليل المشكلة لجزأين يجب حلهما معاً
مثال لغوي	Noun → man | dog	Sentence → NP AND VP
في البحث	نختار فرعاً واحداً	يجب استكشاف كل الفروع

التمثيل البصري

OR Relation:

```text
    p
   ↗ ↖
  q   r

✅ حل ← q إذا أثبتنا
✅ حل ← r أو أثبتنا
```

AND Relation:

```text
    p
   ↗⌒↖   (قوس منحني يربطهما)
  q   r

✅ معاً ← حل r AND q يجب إثبات
```

📌 الجزء الثالث: Hypergraph (الرسم الفائق)

التعريف الرياضي

```text
Hypergraph = (V, E)
V = مجموعة العقد (Vertices)
E = مجموعة الأقواس الفائقة (Hyperarcs)
```

الفرق الأساسي:

الرسم العادي (Graph)	الرسم الفائق (Hypergraph)
القوس يربط **عقدة بعقدة واحدة**	القوس الفائق يربط **عقدة بمجموعة من العقد**
$v_1 \to v_2$	$v_1 \to \{v_2, v_3, v_4\}$
يمثل OR فقط	**يمثل AND و OR**

الربط المباشر

```text
AND Relation = Hyperarc
```

مثال:

```text
لإثبات p، نحتاج: q AND r AND s

في Hypergraph:
Hyperarc: {q, r, s} → p
```

الخاصية المميزة:

- ✅ إذا كانت المجموعة تحتوي على **عقدة واحدة** → OR relation
- ✅ إذا كانت المجموعة تحتوي على **أكثر من عقدة** → AND relation

📌 الجزء الرابع: دليل الأمثلة السريع

التطبيق	نوع البحث	الهيكل المستخدم	الخاصية الفريدة
MACSYMA (تكامل رمزي)	Goal-Driven	**And/Or Graph**	- AND: تحليل المشكلة (decomposition) - OR: استراتيجيات بديلة (تعويض مثلثي أو تجزئة)
Fred Example (Where is Fred?)	**Goal-Driven**	And/Or + Unification	- Subgoals مرتبطة بـ AND - استخدام **Unification** لربط المتغيرات - Backward reasoning
Financial Advisor (Expert Systems)	Goal-Driven	And/Or + User Query	- يسأل المستخدم عند عدم المعرفة - **Backtracking** عند فشل فرع AND
NLP Parsing	Data-Driven / Goal-Driven	**And/Or Hypergraph**	- القواعد النحوية = AND (S → NP AND VP) - المفردات = OR (Noun → man | dog) - Parse Tree = **Solution Tree**

التفاصيل الإضافية

1. MACSYMA:

```text
مثال: ∫(x² + sin(x))dx

AND Node: يجب حل الجزأين
∫x² dx  AND  ∫sin(x) dx

OR Node: لكل جزء، طرق بديلة
∫sin(x) dx → تعويض مثلثي OR جداول
```

2. Fred Example:

```text
Goal: location(fred, X)

:Subgoals (AND)
- is_dog(fred)
- is_good(fred)
- has_owner(fred, Y)
- location(Y, X)

النظام يعمل للخلف ويستخدم Unification
```

3. Financial Advisor:

```text
Rule: inadequate_savings ∧ steady_income → stocks

إذا لم نعرف inadequate_savings:
→ اسأل المستخدم: "ما هو مبلغ التوفير؟"

إذا فشل الفرع:
→ Backtracking لتجربة قاعدة أخرى
```

4. NLP Parsing:

```text
Grammar Rule:
Sentence → NP AND VP  (AND relation)
NP → Det AND Noun    (AND relation)
Noun → man | dog     (OR relation)

Parse Tree للجملة "The man runs":
         Sentence
        /        \
      NP          VP
     / \          |
  Det  Noun      Verb
   |    |         |
  The  man      runs

Parse Tree = Solution Tree من And/Or Graph
```

📌 الجزء الخامس: الحقائق الامتحانية الخمس 🔥

✅ 1. Parse Tree = Solution Tree

```text
شجرة الإعراب (Parse Tree) هي:
- شجرة حل (Solution Tree)
- مستخرجة من And/Or Hypergraph
- الأوراق = Terminals (الكلمات)
- الجذر = Start Symbol (الجملة)
```

✅ 2. Hyperarc ⟺ AND Relation

```text
القوس الفائق (Hyperarc):
- يربط عقدة بـ مجموعة من العقد
- إذا |مجموعة| > 1 → AND relation
- إذا |مجموعة| = 1 → OR relation (قوس عادي)
```

✅ 3. Goal-Driven في Expert Systems

```text
النظام الخبير يستخدم Goal-Driven لأن:
- الهدف واضح (نصيحة استثمارية)
- البيانات تُجمع عند الحاجة (يسأل المستخدم)
- يقلل فضاء البحث (لا نسأل كل الأسئلة)
```

✅ 4. Backtracking عند فشل AND

```text
إذا فشل أحد فروع AND:
- كامل الفرع يفشل (كلهم ضروريون)
- النظام يتراجع (Backtrack)
- يجرب فرع OR آخر (إن وُجد)
```

✅ 5. Unification في Subgoals

```text
في "?Where is Fred":
- X ← Goal: location(fred, X) متغير
- Y ← Subgoal: location(sam, Y) متغير
- Unification يربط: "X = Y = "museum
```

📌 نقاط ذكية للـ MCQ

عن التمثيل:

- ✅ العقدة = حقيقة منطقية
- ✅ القوس = قاعدة استنتاج
- ✅ المسار = برهان منطقي
- ✅ البحث = عملية الاستدلال

عن **AND/OR:**

- ✅ OR: أحدهما كافٍ (أقواس منفصلة)
- ✅ AND: كلهم ضروريون (أقواس مربوطة)
- ✅ OR: نختار فرعاً واحداً
- ✅ AND: يجب استكشاف الكل

عن **Hypergraph:**

- ✅ Hyperarc يربط عقدة بـ **مجموعة** عقد
- ✅ يستخدم لتمثيل AND relations
- ✅ Hypergraph = تعميم للرسم العادي

عن الأمثلة:

- ✅ MACSYMA: تكامل رمزي (And/Or)
- ✅ Fred: Goal-Driven + Unification
- ✅ Expert Systems: Goal-Driven + User Query
- ✅ NLP: Parse Tree = Solution Tree

عن **Parsing:**

- ✅ Grammar Rules → AND (S → NP AND VP)
- ✅ Vocabulary → OR (N → man | dog)
- ✅ Parse Tree من And/Or Graph
- ✅ Terminals = الأوراق

📝أسئلة MCQ امتحانية (20 سؤال)

Q1: In state space representation of logic, a node represents:
- A) Inference rule
- **B) A fact or logical statement** ✅
- C) A path
- D) A proof

Q2: In state space representation of logic, an arc represents:
- A) A fact
- **B) An inference rule (e.g., Modus Ponens)** ✅
- C) A goal
- D) A variable

Q3: A logical proof corresponds to:
- A) A node
- B) An arc
- **C) A solution path** ✅

Q4: q OR r -> p represents:
- A) AND relation
- **B) OR relation** ✅
- C) Both
- D) Neither

Q5: q AND r -> p represents:
- A) OR relation
- **B) AND relation** ✅
- C) XOR relation
- D) Implication only

Q6: In AND relation, to prove p we need:
- A) Prove q or r
- **B) Prove both q and r** ✅
- C) Prove neither
- D) Prove any one

Q7: A Hyperarc connects:
- A) One node to one node
- **B) One node to a set of nodes** ✅
- C) Many nodes to many nodes
- D) No nodes

Q8: Hyperarc is used to represent:
- A) OR relation only
- **B) AND relation** ✅
- C) Neither
- D) Simple arcs

Q9: A Parse Tree in NLP is:
- A) A grammar rule
- **B) A solution tree from And/Or graph** ✅
- C) A vocabulary list

Q10: In MACSYMA, AND nodes represent:

- A) Alternative strategies
- **B) Problem decomposition** ✅
- C) Integration tables

Q11: In MACSYMA, OR nodes represent:

- A) Decomposition
- **B) Alternative solution strategies** ✅
- C) Required steps

Q12: "Where is Fred?" example uses:

- A) Data-Driven only
- **B) Goal-Driven with Unification** ✅
- C) BFS only

Q13: Expert Systems use Goal-Driven because:

- A) Data is always available
- **B) Goal is clear, data acquired as needed** ✅
- C) Memory is unlimited

Q14: When an AND branch fails:

- A) We continue
- **B) The entire branch fails, backtrack** ✅
- C) We stop completely

Q15: In NLP, "Sentence -> NP VP" is:

- A) OR relation
- **B) AND relation** ✅
- C) Optional
- **شرح :**لكي نحصل على جملة، يجب توفر "مبتدأ "و "خبر" معاً.

Q16: In NLP, "Noun -> man | dog" is:

- A) AND relation
- **B) OR relation** ✅
- C) Both

Q17: Leaves of a Parse Tree represent:

- A) Grammar rules
- B) Non-terminals
- **C) Terminals (words)** ✅

Q18: Unification in "Fred" example is used to:

- A) Create rules
- **B) Bind variables (e.g., X = "museum")** ✅
- C) Delete facts

Q19: Hypergraph is a generalization of:

- A) Trees only
- **B) Regular graphs** ✅
- C) Lists

Q20: Financial Advisor asks user when:

- A) Goal is found
- **B) Information is not available** ✅
- C) System starts

🎯 ملخص نهائي (احفظه!)

الربط الأساسي:

```text
Logic Element → State Space Element
------------------------------------
Fact          → Node
Rule          → Arc
Proof         → Solution Path
Known Facts   → Start State
Goal          → Goal Description
```

AND vs OR:

```text
OR: q ∨ r → p
  - أحدهما كافٍ
  - أقواس منفصلة
  - نختار فرعاً واحداً

AND: q ∧ r → p
  - كلهم ضروريون
  - أقواس مربوطة (hyperarc)
  - يجب إثبات الجميع
```

Hypergraph:

```text
Hyperarc: {مجموعة عقد} → عقدة
1 < |مجموعة| إذا → AND relation
```

الأمثلة:

```text
MACSYMA:    And/Or (تكامل)
Fred:       Goal-Driven + Unification
Expert Sys: Goal-Driven + User Query
NLP:        Parse Tree = Solution Tree
```

الحقائق الخمس:

```text
1. Parse Tree = Solution Tree
2. Hyperarc = AND relation
3. Expert Systems = Goal-Driven
4. AND فشل → Backtrack
5. Unification يربط المتغيرات
```